BarStock Solutions - Actividad 2

Grupo 4

Integrantes:

- Morganti Santiago, 43583012, samorganti@uade.edu.ar
- Araujo Nicolás Ezequiel, 46025298, niaraujo@uade.edu.ar
- María Eugenia Alonso, 43442854, mariaeugalonso@uade.edu.ar
- Welz Mathias Gustavo, 40748158, mwelz@uade.edu.ar
- Vazquez Lucas Valentin, 45749665, lucasvazquez@uade.edu.ar

Docente: Bueno Moises Evaristo

Materia: Proceso de Desarrollo de Software

Universidad Argentina De la Empresa

UADE
UNA GRAN UNIVERSIDAD
FUNDACIÓN UADE

Estructura general del sistema

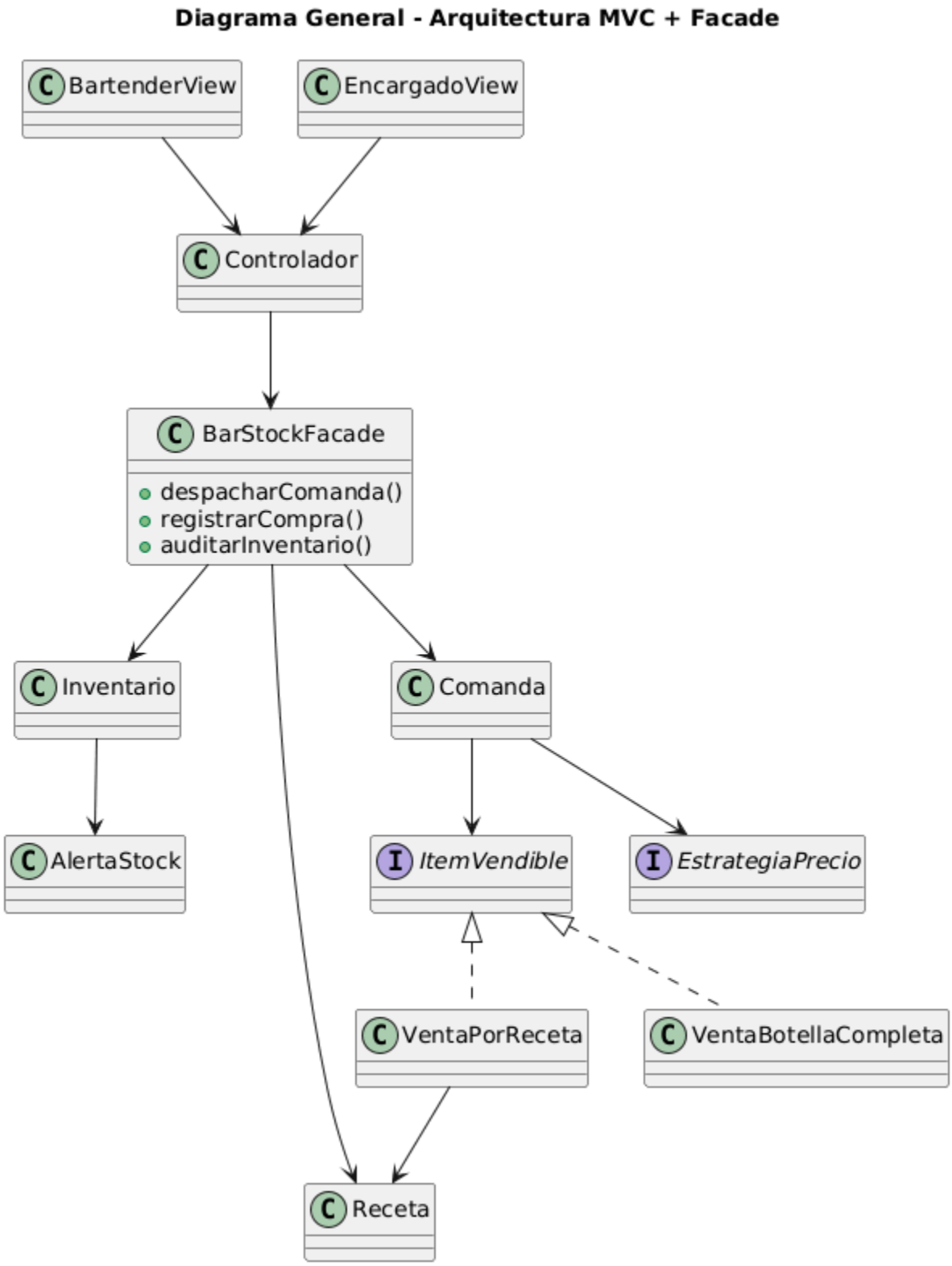
BarStock Solutions es un sistema de gestión y control de inventarios gastronómicos orientado a bares y restaurantes de coctelería de autor. Su objetivo es reemplazar el ABM tradicional de botellas cerradas por un modelo de stock fraccionado multinivel por recetas, que descuenta automáticamente los mililitros exactos de cada insumo al despachar un trago.

Decisión arquitectónica: MVC simple

Se optó por una arquitectura Modelo-Vista-Controlador (MVC) sencilla, sin capas adicionales, dado el alcance del trabajo práctico:

- Modelo: entidades de dominio y lógica de negocio (Insumo, Receta, Inventario, Comanda con sus estados, Venta con su estrategia de precio, AlertaStock como observador).
- Vista: dos interfaces diferenciadas según el rol — la del Bartender (registro ágil de ventas y stock en tiempo real) y la del Encargado (compras, auditoría y reportes).
- Controlador: recibe las acciones de la vista y las traduce en llamadas a BarStockFacade, sin contener lógica de negocio propia.

Esta separación evita mezclar la interfaz de dos perfiles de usuario muy distintos y calza naturalmente con los patrones elegidos: Facade actúa como puerta entre el Controlador y el Modelo, mientras que State, Strategy y Observer viven dentro del Modelo.


Diagrama General - Arquitectura MVC + Facade
BartenderView
EncargadoView
Controlador
BarStockFacade
despacharComanda()
registrarCompra()
auditarInventario()
Inventario
Comanda
AlertaStock
ItemVendible
EstrategiaPrecio
VentaPorReceta
VentaBotellaCompleta
Receta

Patrones de diseño aplicados

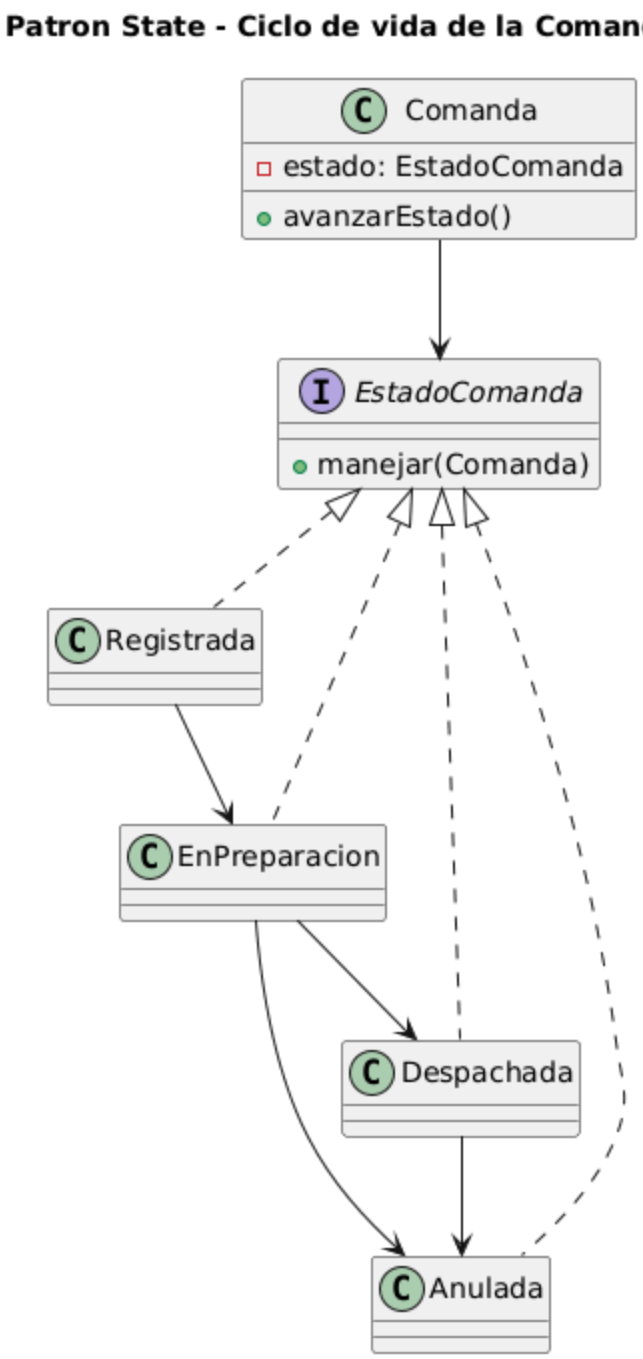
Tipo	Patrones	Aplicación
Comportamiento	State	Estados de una comanda
Comportamiento	Strategy	Cálculo de precios normales, happy hour, etc.
Comportamiento	Observer	Alertas automáticas por bajo stock
Creacional	Simple Factory	Creación del medio de notificación
Estructural	Facade	Interfaz principal para usar el sistema
Estructural	Decorador	Agregar información a los reportes

State

Se utilizó el patrón de comportamiento State para administrar el ciclo de vida de una comanda, ya que tiene varios estados posibles. Una comanda puede encontrarse inicialmente registrada, luego pasar a preparación y finalmente quedar despachada.

Este patrón, evita implementar numerosos condicionales dentro de la clase Comanda. Cada estado se representa mediante una clase independiente, que encapsula el comportamiento correspondiente.

Esto hace que el sistema cumpla con las buenas prácticas, como el fácil mantenimiento y abierto a la extensión (principio SOLID de abierto/cerrado). Esto permite incorporar posteriormente nuevos estados, sin modificar las clases existentes como: Cancelada, Demorada y Entregada.



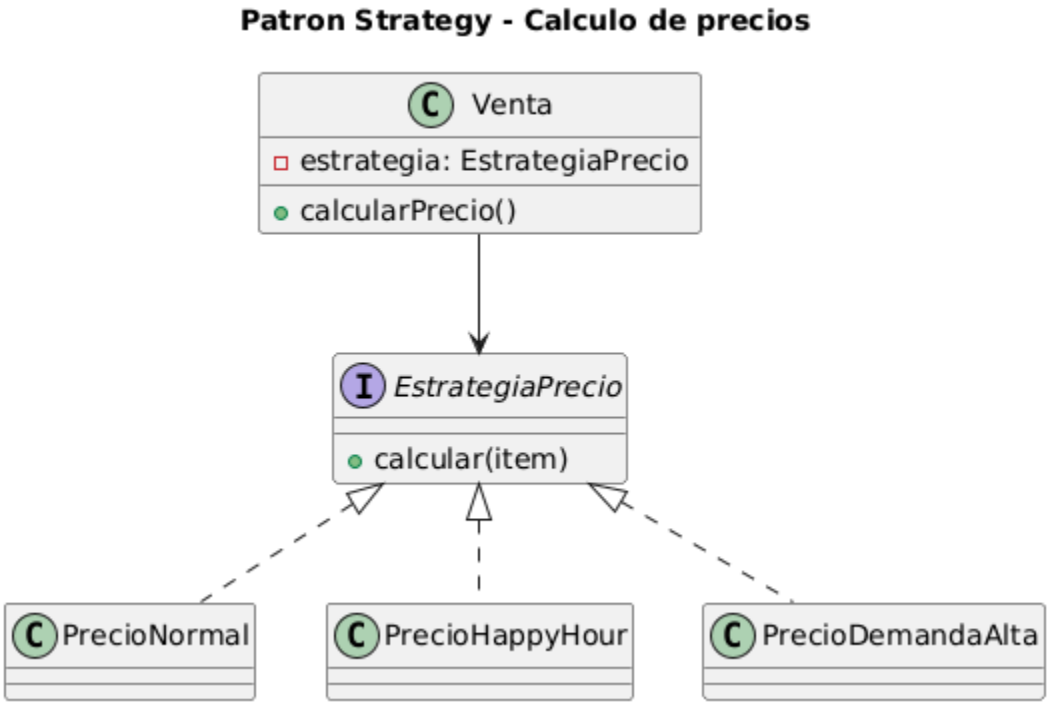
Strategy

Se utilizó Strategy para implementar las diferentes formas de calcular el precio de una venta.

BarStock Solutions necesita manejar distintos tipos de precios, ya que cuenta con promociones como Happy Hour y tragos por temporada.

Por esta razón, se definió la interfaz: EstrategiaPrecio y diferentes implementaciones como: PrecioNormal, PrecioHappyHour y PrecioDemandaAlta

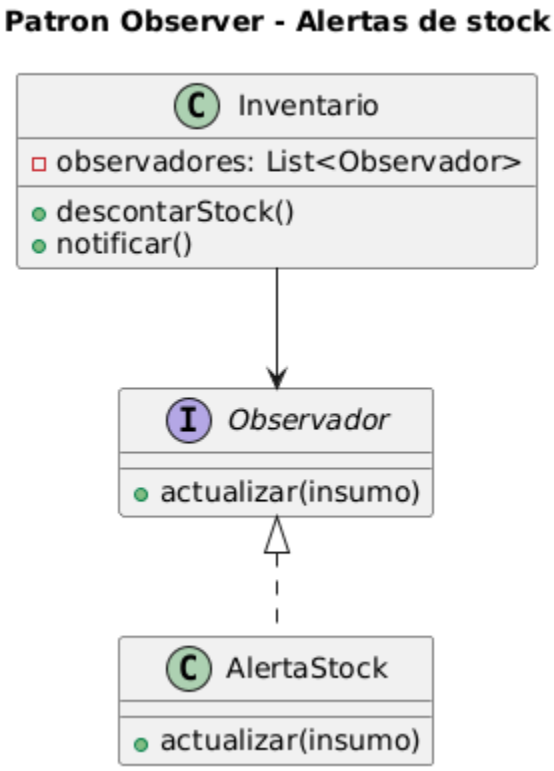
Este patrón permite cambiar en tiempo de ejecución sin modificar las clases responsables de las ventas.



Observer

Se utilizó Observer ya que es necesario poder monitorear el nivel del inventario restante. Cada vez que se consume un ingrediente, Inventario notifica a sus observadores. El observador AlertaStock comprueba si el insumo llegó a su punto de reposición. De esta manera, el

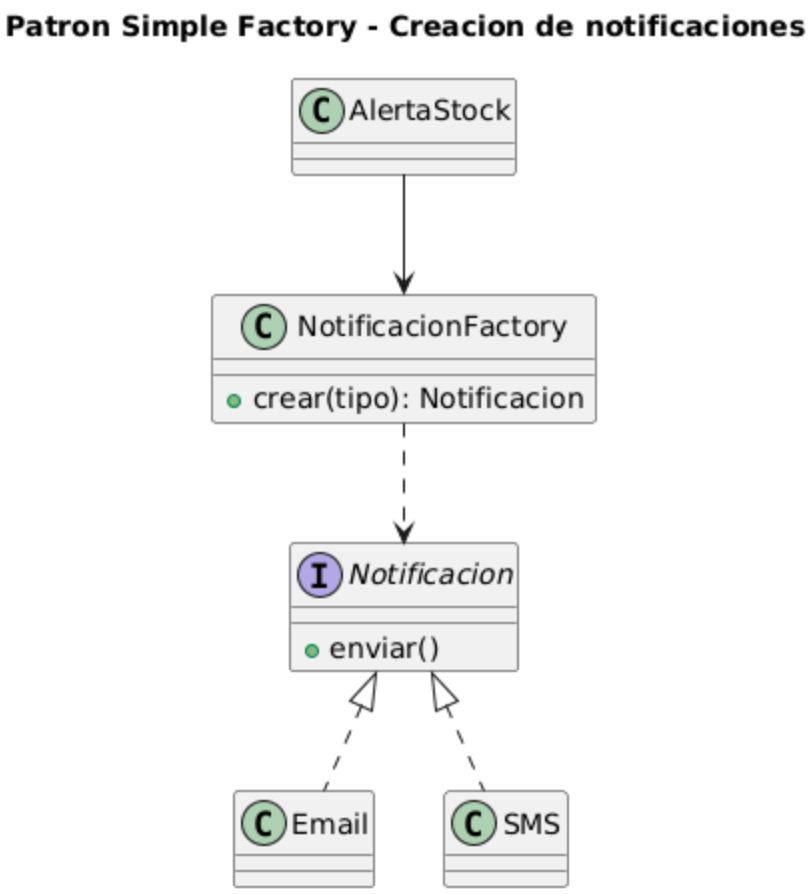
inventario no necesita conocer cómo funciona una notificación.



Simple Factory

Se utilizó Simple Factory para centralizar la creación de medios de notificación. En lugar de distribuir por el sistema instrucciones como: new Email() o new SMS(), se usa: NotificacionFactory.crear(tipo), que determina cuál objeto concreto necesita construirse.

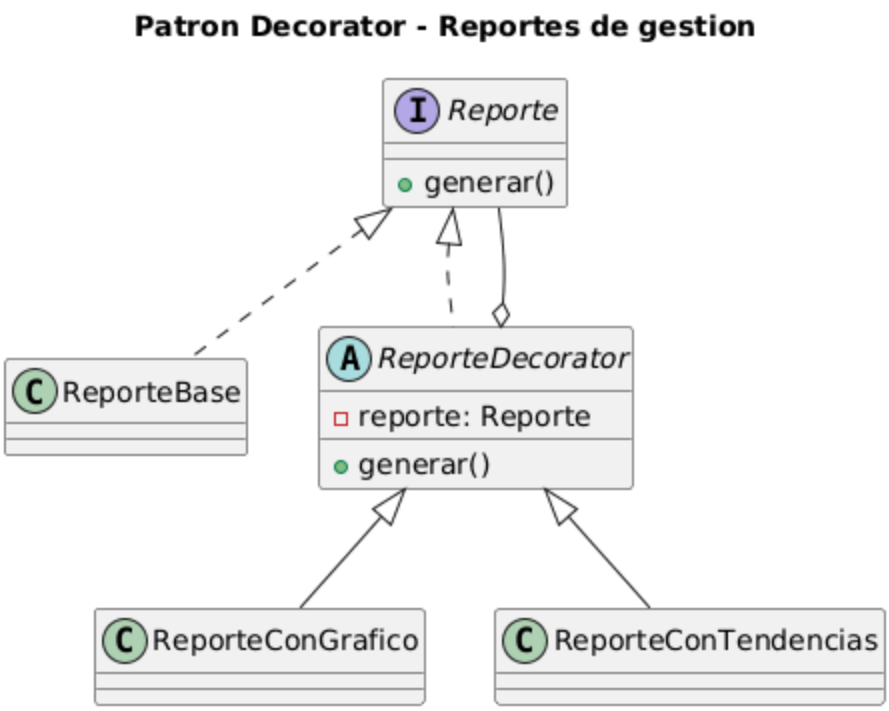
Este patrón, Simple Factory, encapsula en una clase la creación de objetos que poseen una interfaz común.



Decorator

Se utilizó Decorator para construir reportes de gestión para poder mostrar al usuario las ventas realizadas.

Decorator permite añadir responsabilidades dinámicamente sin alterar la clase original, evitando generar múltiples subclases para cada combinación posible de reporte.



Facade

Por último, se utilizó Facade para ofrecer un punto de entrada sencillo al sistema. BarStock contiene múltiples subsistemas, como: Inventario , Recetas, Comandas, etc.

Una interfaz gráfica no debería conocer la implementación interna de todos ellos, por lo que se crea:BarStockFacade.

Este patrón permite reducir el acoplamiento y proporcionar una interfaz unificada al subsistema complejo.

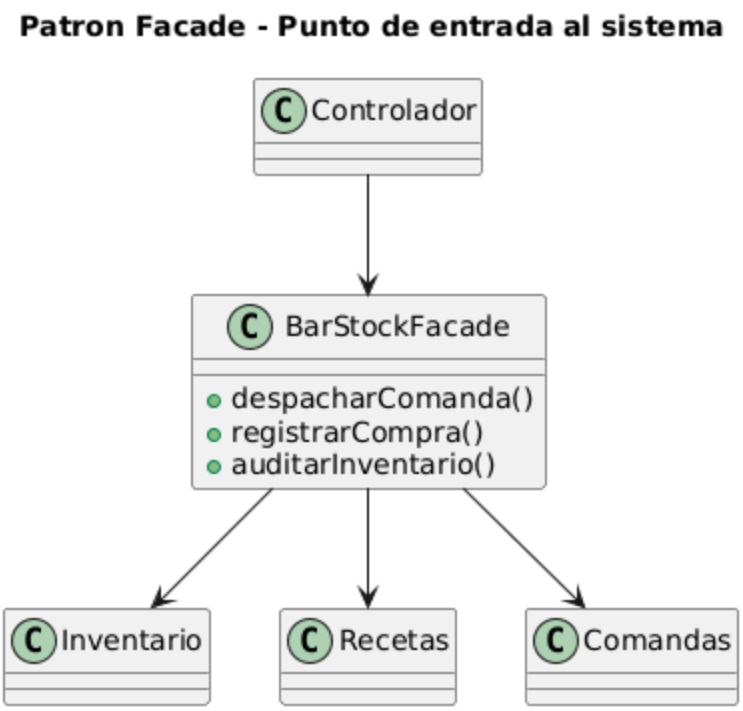
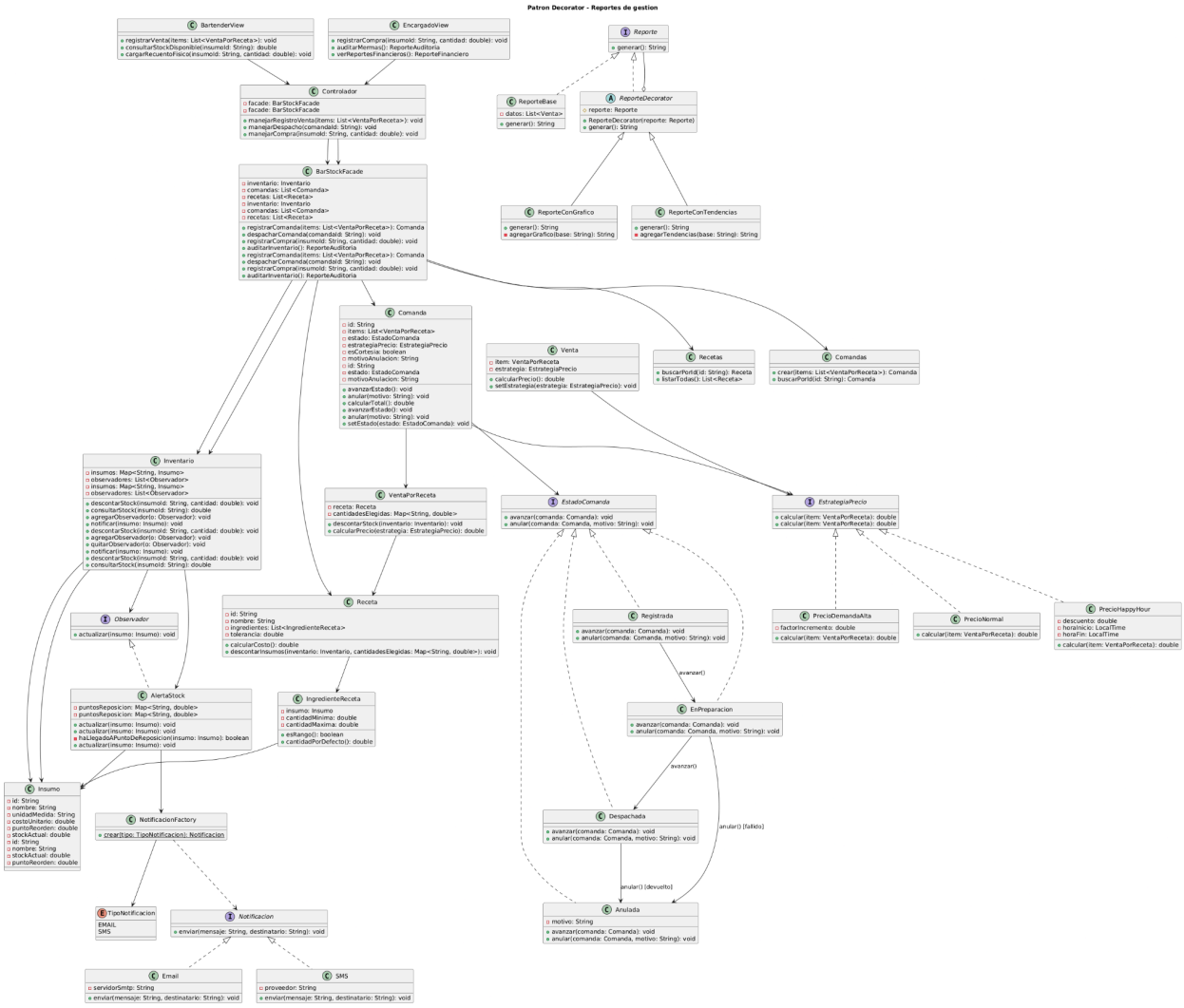


Diagrama General con atributos:

[PlantUML link]


Patron Decorator - Reportes de gestion
BartenderView
+ registrarVenta(items: List<VentaPorReceta>): void
+ consultarStockDisponible(insumoId: String): double
+ cargarRecuentoFisico(insumoId: String, cantidad: double): void
EncargadoView
+ registrarCompra(insumoId: String, cantidad: double): void
+ auditarMermas(): ReporteAuditoria
+ verReportesFinancieros(): ReporteFinanciero
Reporte
+ generar(): String
Controlador
- facade: BarStockFacade
- facade: BarStockFacade
+ manejarRegistroVenta(items: List<VentaPorReceta>): void
+ manejarDespacho(comandaId: String): void
+ manejarCompra(insumoId: String, cantidad: double): void
ReporteBase
- datos: List<Venta>
+ generar(): String
ReporteDecorator
- reporte: Reporte
+ ReporteDecorator(reporte: Reporte)
+ generar(): String
BarStockFacade
- inventario: Inventario
- comandas: List<Comanda>
- recetas: List<Receta>
- inventario: Inventario
- comandas: List<Comanda>
- recetas: List<Receta>
+ registrarComanda(items: List<VentaPorReceta>): Comanda
+ despacharComanda(comandaId: String): void
+ registrarCompra(insumoId: String, cantidad: double): void
+ auditarInventario(): ReporteAuditoria
+ registrarComanda(items: List<VentaPorReceta>): Comanda
+ despacharComanda(comandaId: String): void
+ registrarCompra(insumoId: String, cantidad: double): void
+ auditarInventario(): ReporteAuditoria
ReporteConGrafico
+ generar(): String
- agregarGrafico(base: String): String
ReporteConTendencias
+ generar(): String
- agregarTendencias(base: String): String
Comanda
- id: String
- items: List<VentaPorReceta>
- estado: EstadoComanda
- estrategiaPrecio: EstrategiaPrecio
- esCortesia: boolean
- motivoAnulacion: String
- id: String
- estado: EstadoComanda
- motivoAnulacion: String
+ avanzarEstado(): void
+ anular(motivo: String): void
+ calcularTotal(): double
+ avanzarEstado(): void
+ anular(motivo: String): void
+ setEstado(estado: EstadoComanda): void
Venta
- item: VentaPorReceta
- estrategia: EstrategiaPrecio
+ calcularPrecio(): double
+ setEstrategia(estrategia: EstrategiaPrecio): void
Recetas
+ buscarPorId(id: String): Receta
+ listarTodas(): List<Receta>
Comandas
+ crear(items: List<VentaPorReceta>): Comanda
+ buscarPorId(id: String): Comanda
Inventario
- insumos: Map<String, Insumo>
- observadores: List<Observador>
- insumos: Map<String, Insumo>
- observadores: List<Observador>
+ descontarStock(insumoId: String, cantidad: double): void
+ consultarStock(insumoId: String): double
+ agregarObservador(o: Observador): void
+ notificar(insumo: Insumo): void
+ descontarStock(insumoId: String, cantidad: double): void
+ agregarObservador(o: Observador): void
+ quitarObservador(o: Observador): void
+ notificar(insumo: Insumo): void
+ descontarStock(insumoId: String, cantidad: double): void
+ consultarStock(insumoId: String): double
VentaPorReceta
- receta: Receta
- cantidadesElegidas: Map<String, double>
+ descontarStock(inventario: Inventario): void
+ calcularPrecio(estrategia: EstrategiaPrecio): double
EstadoComanda
+ avanzar(comanda: Comanda): void
+ anular(comanda: Comanda, motivo: String): void
EstrategiaPrecio
+ calcular(item: VentaPorReceta): double
+ calcular(item: VentaPorReceta): double
Observador
+ actualizar(insumo: Insumo): void
Receta
- id: String
- nombre: String
- ingredientes: List<IngredienteReceta>
- tolerancia: double
+ calcularCosto(): double
+ descontarInsumos(inventario: Inventario, cantidadesElegidas: Map<String, double>): void
Registrada
+ avanzar(comanda: Comanda): void
+ anular(comanda: Comanda, motivo: String): void
PrecioDemandaAlta
- factorIncremento: double
+ calcular(item: VentaPorReceta): double
PrecioNormal
+ calcular(item: VentaPorReceta): double
PrecioHappyHour
- descuento: double
- horaInicio: LocalTime
- horaFin: LocalTime
+ calcular(item: VentaPorReceta): double
avanzar()
AlertaStock
- puntosReposicion: Map<String, double>
- puntosReposicion: Map<String, double>
+ actualizar(insumo: Insumo): void
+ actualizar(insumo: Insumo): void
- haLlegadoAPuntoDeReposicion(insumo: Insumo): boolean
+ actualizar(insumo: Insumo): void
IngredienteReceta
- insumo: Insumo
- cantidadMinima: double
- cantidadMaxima: double
+ esRango(): boolean
+ cantidadPorDefecto(): double
EnPreparacion
+ avanzar(comanda: Comanda): void
+ anular(comanda: Comanda, motivo: String): void
avanzar()
anular() [fallido]
Insumo
- id: String
- nombre: String
- unidadMedida: String
- costoUnitario: double
- puntoReorden: double
- stockActual: double
- id: String
- nombre: String
- stockActual: double
- puntoReorden: double
NotificacionFactory
+ crear(tipo: TipoNotificacion): Notificacion
Despachada
+ avanzar(comanda: Comanda): void
+ anular(comanda: Comanda, motivo: String): void
anular() [devuelto]
TipoNotificacion
EMAIL
SMS
Notificacion
+ enviar(mensaje: String, destinatario: String): void
Anulada
- motivo: String
+ avanzar(comanda: Comanda): void
+ anular(comanda: Comanda, motivo: String): void
Email
- servidorSmtp: String
+ enviar(mensaje: String, destinatario: String): void
SMS
- proveedor: String
+ enviar(mensaje: String, destinatario: String): void